Classical counterpart

Tong Shen

Abstract

This is the classical counterpart of the Distributed Quantum Linear Solver. We will also briefly introduce the difference of the quantum design.

1 Problem formulation

We are interested in finding the least square solution to a large linear system in the form $Ax = b$. Suppose the linear system of equations can be partitioned into:

$$\begin{bmatrix} A_{11} & \cdots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \cdots & A_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix},$$

where m and n are two positive integers, each A_{ij} is a square submatrix in $\mathbb{R}^{d\times d}$ of the same size, and each subvector $b_i \in \mathbb{R}^d$ shares the same number of rows as A_{ij}. To reduce computational complexity, we use a network of mn agents to collaboratively compute a least square solution. Each agent is assigned a submatrix A_{ij} and a vector $b_{ij} \in \mathbb{R}^d$ such that $\sum_{j=1}^{n} b_{ij} = b_i$. The goal of each agent is to compute an $x_j^* \in \mathbb{R}^d$ such that the stacked vector

$$x^* = \begin{bmatrix} x_1^* \\ x_2^* \\ \cdots \\ x_n^* \end{bmatrix} \in \mathbb{R}^{nd} \tag{1}$$

forms a least square solution for the system, namely

$$x^* \in \arg\min_{x\in\mathbb{R}^{dn}} \|Ax - b\|^2 \tag{2}$$

To present our algorithm, we need the following concepts. For each agent $[\![ij]\!]$, agent $[\![ik]\!]$ is called a row-neighbor of agent $[\![ij]\!]$ if agent $[\![ij]\!]$ can receive information from agent $[\![ik]\!]$. Similarly, agent $[\![kj]\!]$ is called a column-neighbor of agent $[\![ij]\!]$ if agent $[\![ij]\!]$ can receive information from agent $[\![kj]\!]$. We use $\mathcal{N}_{ij}^{\text{row}}$ and $\mathcal{N}_{ij}^{\text{col}}$ to denote the row- and column-neighbor sets of agent $[\![ij]\!]$, respectively.

For presentation simplicity, we assume that each agent is always a row- and column-neighbor of itself, so $\mathcal{N}_{ij}^{\text{row}}$ and $\mathcal{N}_{ij}^{\text{col}}$ are always nonempty. These neighbor relationships can be described by graphs. For each index $i \in \{1, \ldots, m\}$, we define a row-neighbor graph $\mathbb{G}_i^{\text{row}}$ with n vertices to represent the neighbor relationships among agents $[\![ij]\!], j \in \{1, \ldots, n\}$ in that (k, j) is an arc (i.e., a directed edge) in $\mathbb{G}_i^{\text{row}}$ whenever agent $[\![ik]\!]$ is a neighbor of agent $[\![ij]\!]$. Similarly, for each index $j \in \{1, \ldots, n\}$, we define a column-neighbor graph $\mathbb{G}_j^{\text{col}}$ with m vertices to represent the neighbor relationships among agents $[\![ij]\!], i \in \{1, \ldots, m\}$ in that (k, i) is an arc in $\mathbb{G}_j^{\text{col}}$ whenever agent $[\![kj]\!]$ is a neighbor of agent $[\![ij]\!]$. Note that these definitions imply that the directions of the arcs represent the directions of information flow. Since each agent is always its own row- and column-neighbor, all row- and column-neighbor graphs have self-arcs at every vertex. The following assumption is imposed throughout the paper.

Assumption 1. *All row-neighbor graphs $\mathbb{G}_i^{row}, i \in \{1, \ldots, m\}$ are bidirectional and connected. All column-neighbor graphs $\mathbb{G}_j^{col}, j \in \{1, \ldots, n\}$ are identical, bidirectional, and connected.*

A bidirectional graph can be simplified as a simple undirected graph, in that each pair of directed edges between two vertices is replaced by a single undirected edge. Under the above assumption, we let $\mathbb{G}$ denote the common graph of the column-neighbor graphs $\mathbb{G}_j^{\text{col}}, j \in \{1, \ldots, n\}$, which is undirected and connected. With this, the notations $\mathcal{N}_{ij}^{\text{col}}, \mathbb{G}_i^{\text{row}}, \mathcal{N}_{ij}^{\text{row}}$ can be simplified as $\mathcal{N}_i, \mathbb{G}_i, \mathcal{N}_{ij}$ respectively, without causing confusion. From the preceding, all graphs $\mathbb{G}$ and $\mathbb{G}_i, i \in \{1, \ldots, m\}$ have self-loops at each vertex.

2 Classical algorithm for block partition

[type down also the distributed problem formulation and algorithm for both Adam and gradient tracking; it is for later comparison purpose. after you finish your algorithm, you will need to comment why it is a combination of Adam and gradient tracking; e.g. showing if xxx, then your algorithm simplifies to Adam.]

[always make sure you refer to a solid reference. Adam is a popular optimization technique, there must be a well cited formal publication for distributed Adam in a decent/top venue; be cautious about informal version like arXiv ones, especially those a while ago; many good papers have online informal/arXiv versions, always check and follow the formal published versions. never build your research on unreliable resources]

2.1 Mathematical foundation

We now discuss how we construct the objective function of our distributed problem. Consider finding the least square solution for the linear system with block partition 3.1. First, we define the pure column partition problem for the $i-$th row using the following notations:

$$A_i = \begin{bmatrix} A_{i1} & A_{i2} & \cdots & A_{in} \end{bmatrix}, \quad \mathcal{N} = \{1, 2, \cdots, n\}$$

Consider the problem of pure column partition:

$$A_i x = \sum_{j=1}^{n} A_{ij} x_j = b_i,\ x_j \in \mathbb{R}^d,\ j \in \mathcal{N}, \tag{3}$$

where $b_i \in \mathbb{R}^d$, and $x = [x_1^\top, \cdots, x_n^\top]^\top$. We want to search for the least square solution $x^* \in \arg\min_x \|A_i x - b_i\|^2$

Let $\bar{A}_i = \text{diag}\{A_{i1}, \cdots, A_{in}\}$ and $\bar{b}_i = [b_{i1}^\top, \cdots, b_{in}^\top]^\top$, where $\sum_{j=1}^n b_{ij} = b_i,\ b_{ij} \in \mathbb{R}^d$. Let $z_i = [z_{i1}^\top, \cdots, z_{in}^\top]^\top$. The agents are connected by $\mathbb{G}_i^{row} = (\mathcal{N}, \mathcal{E}_i)$, where $\mathcal{E}_i$ is the set of communication links. Let $W^i = [W_{k,j}^i]$ be a weight matrix such that $W_{k,j}^i = 1$ if $(k,j) \in \mathcal{E}_i$, and $W_{k,j}^i = 0$ otherwise, so that W^i respects the communication graph $\mathbb{G}_i^{row}$. Let L^i be the Laplacian matrix for row i: $L^i = D^i - W^i$. Then, we have the following result:

Lemma 1. *If there exists a pair $(x^*, z_i^*) \in \mathbb{R}^{nd}$ such that $(x^*, z_i^*) \in \arg\min_{(x,z_i)} \|\bar{A}_i x - \bar{b}_i - \bar{L}^i z_i\|^2$, then $x^* \in \arg\min_x \|A_i x - b_i\|^2$.*

where $\bar{L}^i = L^i \otimes I_d$. For block partition, we have the following lemma:

Lemma 2. *(with some connectivity condition) $x^* \in \mathbb{R}^K \in \arg\min_x \sum_i \|A_i x - b_i\|^2$ if and only if $x^* \in \arg\min_x \|Ax - b\|^2$.*

Thus, block partition problem can be formulated as

$$\begin{aligned} x^* \in \arg\min_x \|Ax - b\|^2 \quad &\Longleftrightarrow \quad x^* \in \arg\min_x \textstyle\sum_{i=1}^m \|A_i x - b_i\|^2 && \textit{(From Lemma 2)} \\ &\Longleftarrow \quad (x^*, \{z_1^*, \cdots, z_m^*\}) \in \arg\min_{(x,\{z_1,\cdots,z_m\})} \textstyle\sum_{i=1}^m \|\bar{A}_i x - \bar{b}_i - \bar{L}_i z_i\|^2 && \textit{(From Lemma 1)} \end{aligned}$$

[The inference is problematic because minimizing a sum does not imply minimizing each term jointly. Use the original lemma 2 statement, i.e., the statement with (x, z^*).]

TS: I am a little confused about the notation z^*. For a pure column partition problem, we can have x^* and z^*, as lemma 1. But for a block partition problem with m rows, or we can say m column partition problems, we actually have m different z^*, so I use the notation z_i and z_i^*.Agents do not transmit z_i to its column neighbors.

Zeru: Based on the original lemma statement, even for a m column partition problem, there is a single z^* such that if (x^*, z^*) form an optimal solution to $\min \sum_j \|\bar{A}_j x - \bar{b}_j - \bar{L} z\|^2$, then $x^* \in \arg\min_x \|Ax - b\|^2$. It says nothing about m different z^*.

However, during my checking today, I realized that the original proof is not valid, and I just wrote a new proof. Fortunately, the new statement has to use $\{z_i\}$ for each row i. Therefore, you can keep the current structure that agents do not transmit z_i to its column neighbors. You can check my new lemma (lemma 5) and proof in the overleaf "column matrix for Youngin".

The objective function to minimize is

$$f = \sum_{i=1}^{m} \|\bar{A}_i x - \bar{b}_i - \bar{L}_i z_i\|^2 = \sum_{i=1}^{m}\sum_{j=1}^{n} \|A_{ij}x_j - b_{ij} - \sum_{[ik]\in\mathcal{N}_{ij}} L^i_{jk} z_{ik}\|^2 = \sum_{i=1}^{m}\sum_{j=1}^{n} f_{ij}. \tag{4}$$

Each agent $[\![ij]\!]$ is assigned two local variables, x_{ij} [inconsistency of notation, x_{ij} is not introduced] TS: The introduction is in the algorithm part, and I copy it here: Conceptually, x_{ij} is the local copy of the solution estimate $x_j \in \mathbb{R}^d$, which is the component of the stacked solution vector as in 33 and z_{ij}. Agents in the same column need to reach consensus on their x_{ij} for a global solution x^*, so variable x_{ij} and corresponding gradient information is transmitted among column neighbors. Agents in the same row need to coordinate to solve a column partition problem, so variable z_{ij} and corresponding gradient information is transmitted among row neighbors.

2.2 Techniques

Our algorithm combines two techniques in distributed optimization and machine learning respectively.

DIGing(gradient tracking) DIGing [1] is an algorithm design for following distributed convex optimization problem:

$$\min_{x\in\mathbb{R}^d} f(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x) \tag{5}$$

where function f_i are convex and continuously differentiable. Each agent i has access to the local objective function f_i and the local copy of the variable x, denoted as $x_i \in \mathbb{R}^d$.

Consider the distributed gradient descent (DGD):

$$\mathbf{x}(k+1) = \mathbf{W}\mathbf{x}(k) - \alpha\nabla\mathbf{f}(\mathbf{x}(k)) \tag{6}$$

where $\mathbf{W}$ is a doubly stochastic mixing matrix and $\alpha > 0$ is a fixed step-size. DGD will exhibit undesirable behavior since the local gradient direction $\nabla f_i(x_i(k))$ is not necessarily zero at the solution point x^*, and will move $x_i(k+1)$ away from x^*. DIGing introduce a surrogate direction denoted $\mathbf{y}(k)$ and updated as:

$$\mathbf{y}(k+1) = \mathbf{W}\mathbf{y}(k) + \nabla\mathbf{f}(\mathbf{x}(k+1)) - \nabla\mathbf{f}(\mathbf{x}(k)) \tag{7}$$

with initialization $\mathbf{y}(0) = \nabla\mathbf{f}(\mathbf{x}(0))$. The variable $\mathbf{y}(k)$ can be seen as a dynamic approximation of the average gradient $\frac{1}{n}\mathbf{1}\mathbf{1}^\top\nabla\mathbf{f}(\mathbf{x}(k))$.

Looking at an individual agent i, the initialization of DIGing uses an arbitrary $x_i(0) \in \mathbb{R}^p$ and sets $y_i(0) = \nabla f_i(x_i(0))$ for all $i = 1, \ldots, n$. At each iteration k, every agent i maintains two vectors,

namely, $x_i(k), y_i(k) \in \mathbb{R}^p$, which are updated as follows:

$$x_i(k+1) = \sum_{j \in \mathcal{N}_i(k)} W_{ij}(k) x_j(k) - \alpha y_i(k),$$
$$y_i(k+1) = \sum_{j \in \mathcal{N}_i(k)} W_{ij}(k) y_j(k) + \nabla f_i(x_i(k+1)) - \nabla f_i(x_i(k)),$$

where $\mathcal{N}_i(k)$ is the set of all neighbors of agent i at time k. At every iteration k, each agent i sends its current solution estimate $x_i(k)$ and average gradient estimate $y_i(k)$ to all of its neighbors $\mathcal{N}_i(k)$ while receiving all of its neighbors' solution estimates $x_j(k)$ and average gradient estimates $y_j(k), \forall j \in \mathcal{N}_i(k)$. Then, each agent i updates its vector $x_i(k+1)$ by mixing its own $x_i(k)$ and the neighbors' copies $x_j(k)$, for $j \in \mathcal{N}_i(k)$, with specific weights, and adjusting along the direction of $-y_i(k)$. Also, each agent i updates its direction $y_i(k+1)$ by mixing its own $y_i(k)$ and the neighbors' directions $y_j(k)$, for $j \in \mathcal{N}_i(k)$ with specific weights, and by taking into account only the new information contained in the most recent gradient evaluation, as captured in the gradient-difference term $\nabla f_i(x_i(k+1)) - \nabla f_i(x_i(k))$.

Adam Adam [2] (Adaptive Moment Estimation) is designed for first-order gradient-based optimization of stochastic objective functions. It combines the advantages of Momentum and RMSprop(Root Mean Square Propagation) to adaptively adjust learning rates during optimization.

Consider minimize the objective function $f(x)$, the Adam optimizer maintains a first momentum vector $a(t)$ and a second momentum vector $b(t)$, both initialized to be zero and have the following update rules:

$$a(t+1) = \beta_1 a(t) + (1-\beta_1)\nabla f(x(t)) \tag{8}$$
$$b(t+1) = \beta_2 b(t) + (1-\beta_2)(\nabla f(x(t)))^{\odot 2} \tag{9}$$
$$\tag{10}$$

Here $(\nabla f(x^{(t)}))^{\odot 2}$ denotes the element-wise square operation, which means that each element in the gradient is multiplied by itself. Next, the x parameter is updated using the adaptive stepsize:

$$\eta(t+1) = \eta \frac{\sqrt{1-\beta_2^{t+1}}}{1-\beta_1^{t+1}} \tag{11}$$
$$x(t+1) = x(t) - \eta(t+1)\frac{a(t+1)}{\sqrt{b(t+1)}+\epsilon} \tag{12}$$

2.3 Algorithm

We now give our distributed algorithm for block partition. First, the local objective function and aggregate objective function is as follows:

$$f_{ij}(x_i, z_i) := \|A_{ij}x_{ij} - b_{ij} - \sum_{k\in\mathcal{N}_{ij}} L^i_{jk} z_{ik}\|^2, \tag{13}$$

$$f(x_i, \{z_i\}) := \sum_{i=1}^{m} \|\bar{A}_i x_i - \bar{b}_i - \bar{L}^i z_i\|^2 = \sum_{i=1}^{m}\sum_{j=1}^{n} f_{ij}(x_i, z_i), \tag{14}$$

where $\bar{L}^i := L^i \otimes I_d$, $x_i = [x_{i1}^\top, \ldots, x_{in}^\top]^\top$ and $z_i = [z_{i1}^\top, \ldots, z_{in}^\top]^\top$. The optimization goal is to minimize the aggregate objective function f, meanwhile all the x_{ij} in the same column should reach consensus under some tolerance bound, namely

$$\forall j \in \{1, \cdots, n\}, \quad \forall i, k \in \{1, \cdots, m\}, \quad \|x_{ij} - x_{kj}\| \le \epsilon_{\text{con}}. \tag{15}$$

Each agent $[\![ij]\!]$ maintains three variables: $x_{ij} \in \mathbb{R}^d, y_{ij} \in \mathbb{R}^d, z_{ij} \in \mathbb{R}^d$. Conceptually, x_{ij} is the local copy of the solution estimate $x_j \in \mathbb{R}^d$, which is the component of the stacked solution vector as in 33. Variable y_{ij} is the surrogate direction for tracking the average gradient among rows, and substitutes the gradient $\nabla f(x(t))$ in Adam update rules 8. Variable z_{ij} is the messenger variable designed to solve column partition problem.

Each agent has its local objective function, whose input includes static variable A_{ij}, b_{ij} and variable $x_{ij}(t)$ and $z_{ij}(t)$, plus the messenger $z_{ik}(t), [\![ik]\!] \in \mathcal{N}_{ij}$ from its row neighbors. During each iteration, inside each row, each agent $[\![ij]\!]$ first send its messenger variable $z_{ij}(t)$ to its row neighbors in $\mathcal{N}_{ij}$ while receiving all the gradient regarding $z_{ij}(t)$ from its row neighbors, namely $\frac{\partial f_{ik}}{\partial z_{ij}(t)}$ for $[\![ik]\!] \in \mathcal{N}_{ij}$, and then sum up to get the aggregate gradient of $z_{ij}(t)$ for next update step:

$$h_{ij}(t) = \frac{\partial f}{\partial z_{ij}(t)} = \sum_{k\in\mathcal{N}_{ij}} \frac{\partial f_{ik}}{\partial z_{ij}(t)}. \tag{16}$$

Also, inside each column, each agent $[\![ij]\!]$ send its current solution estimation $x_{ij}(t)$ and average gradient estimate $y_{ij}(t)$ to its column neighbors $[\![lj]\!] \in \mathcal{N}_i$ while receiving all of its neighbors' solution estimates $x_{lj}(t)$ and average gradient estimates $y_{lj}(t)$. Then, each agent mixes its own $x_{ij}(t)$ and the column neighbor's $x_{lj}(t)$ using weight matrix W, and perform update using $y_{ij}(t)$. Also, each agent update its variable $y_{ij}(t)$ by mixing its own $y_{ij}(t)$ and the column neighbor's $y_{lj}(t)$ using weight matrix W, plus the gradient tracking term $g^x_{ij}(t+1) - g^x_{ij}(t)$, where

$$g^x_{ij}(t{+}1) := \nabla_{x_{ij}} f_{ij}(x_i(t{+}1), z_i(t{+}1)) = 2A_{ij}^\top \left(A_{ij}x_{ij}(t{+}1) - b_{ij} - \sum_{k\in\mathcal{N}_{ij}} L^i_{jk} z_{ik}(t{+}1) \right). \tag{17}$$

After having update direction y_{ij} and h_{ij} for variable x_{ij} and z_{ij} respectively, we use the Adam update rule for one step iteration. We initialize the first momentum vector $a_x(0), a_z(0)$ and second momentum vector $b_x(0), b_z(0)$ all from zero, then perform the following rules:

$$a_{x,ij}(t+1) = \beta_1 a_{x,ij}(t) + (1-\beta_1) y_{ij}(t) \qquad b_{x,ij}(t+1) = \beta_2 b_{x,ij}(t) + (1-\beta_2)(y_{ij}(t))^{\odot 2} \tag{18}$$

$$a_{z,ij}(t+1) = \beta_1 a_{z,ij}(t) + (1-\beta_1) h_{ij}(t) \qquad b_{z,ij}(t+1) = \beta_2 b_{z,ij}(t) + (1-\beta_2)(h_{ij}(t))^{\odot 2} \tag{19}$$

$$\eta(t+1) = \eta \frac{\sqrt{1-\beta_2^{t+1}}}{1-\beta_1^{t+1}} \tag{20}$$

$$z_{ij}(t+1) = z_{ij}(t) - \eta(t+1) \frac{a_{z,ij}(t+1)}{\sqrt{b_{z,ij}(t+1)} + \epsilon} \tag{21}$$

$$x_{ij}(t+1) = \sum_{k \in \mathcal{N}_i} W_{ik} x_{kj}(t) - \eta(t+1) \frac{a_{x,ij}(t+1)}{\sqrt{b_{x,ij}(t+1)} + \epsilon} \tag{22}$$

$$y_{ij}(t+1) = \sum_{k \in \mathcal{N}_i} W_{ik} y_{kj}(t) + g_{ij}^x(t+1) - g_{ij}^x(t) \tag{23}$$

Here β_1, β_2 are the hyperparameter governing the first and second moment updates. η is the initial constant stepsize.

A detailed description of the proposed algorithm is presented as follows.

Initialization. Agent $[\![ij]\!]$ chooses arbitrary $x_{ij}(0) \in \mathbb{R}^d$ and $z_{ij}(0) \in \mathbb{R}^d$. It initializes the first/second moment variables for Adam: $a_{x,ij}(0) = 0, b_{x,ij}(0) = 0, a_{z,ij}(0) = 0, b_{z,ij}(0) = 0$. It sends $z_{ij}(0)$ to its row neighbors $\{[\![ik]\!] : k \in \mathcal{N}_{ij}\}$ and receives $\{z_{ik}(0)\}_{k \in \mathcal{N}_{ij}}$. It initializes the tracker by one local gradient evaluation: $y_{ij}(0) := \nabla_{x_{ij}} f_{ij}(x_i(0), z_i(0)) = g_{ij}^x(0)$.

Iteration. At each time $t = 0, 1, 2, \ldots$, every agent $[\![ij]\!]$ performs the following steps.

1. **Row Communication:**
   (a) **Computing:** Agent $[\![ij]\!]$ computes gradients $\{\frac{\partial f_{ij}}{\partial z_{ik}(t)}\}_{k \in \mathcal{N}_{ij}}$.
   (b) **Transmitting:** Agent $[\![ij]\!]$ sends $\{\frac{\partial f_{ij}}{\partial z_{ik}(t)}\}$ to its row neighbors $\{[\![ik]\!] : k \in \mathcal{N}_{ij}\}$ and receives $\{\frac{\partial f_{ik}}{\partial z_{ij}(t)}\}_{k \in \mathcal{N}_{ij}}$.
   (c) **Updating:** Agent $[\![ij]\!]$ first computes aggregate gradient $h_{ij}(t)$ according to (16), then updates the Adam moments and the bias-corrected stepsize $a_{z,ij}(t+1)$, $b_{z,ij}(t+1)$, $\eta(t+1)$ according to (19)–(20), respectively, and finally updates $z_{ij}(t+1)$ according to (21).
   (d) **Transmitting:** Agent $[\![ij]\!]$ sends $z_{ij}(t+1)$ to its row neighbors $\{[\![ik]\!] : k \in \mathcal{N}_{ij}\}$ and receives $\{z_{ik}(t+1)\}_{k \in \mathcal{N}_{ij}}$.
2. **Column Communication:**

(a) **Transmitting:** Agent $[\![ij]\!]$ sends $(x_{ij}(t), y_{ij}(t))$ to its column neighbors $\{[\![kj]\!] : k \in \mathcal{N}_i\}$ and receives $\{(x_{kj}(t), y_{kj}(t))\}_{k\in\mathcal{N}_i}$.

(b) **Updating:** Agent $[\![ij]\!]$ first updates the Adam moments $a_{x,ij}(t+1)$, $b_{x,ij}(t+1)$ according to (18), then updates $x_{ij}(t+1)$ according to (22). After computing $g^x_{ij}(t+1)$ according to (17), it finally updates $y_{ij}(t+1)$ according to (23).

2.4 Analysis

2.4.1 Interpretation of the lifted row objective: consensus vs. disagreement

This subsection explains the role of the messenger variable z_i through a simple consensus–disagreement decomposition of the stacked residual within each row.

Fix a row $i \in \{1, \dots, m\}$. Define the local (block) residual at agent $[\![ij]\!]$ as

$$r_{ij}(x) \;:=\; A_{ij}x_j - b_{ij} \in \mathbb{R}^d, \qquad j = 1, \dots, n,$$

and stack them into

$$r_i(x) \;:=\; \bar{A}_i x - \bar{b}_i = \begin{bmatrix} r_{i1}(x) \\ \vdots \\ r_{in}(x) \end{bmatrix} \in \mathbb{R}^{nd}.$$

Let the row-average of these residual blocks be

$$\bar{r}_i(x) \;:=\; \frac{1}{n}\sum_{j=1}^{n} r_{ij}(x) \in \mathbb{R}^d.$$

The corresponding consensus vector is obtained by repeating the same block $\bar{r}_i(x)$ across all agents in the row:

$$r_i^{\mathrm{con}}(x) \;:=\; \mathbf{1} \otimes \bar{r}_i(x) = \begin{bmatrix} \bar{r}_i(x) \\ \vdots \\ \bar{r}_i(x) \end{bmatrix}.$$

Intuitively, $r_i^{\mathrm{con}}(x)$ is what the row residual would look like if every agent in the row agreed on the same residual value.
Define the disagreement part as the deviation from the average:

$$r_i^{\mathrm{dis}}(x) \;:=\; r_i(x) - r_i^{\mathrm{con}}(x) = \begin{bmatrix} r_{i1}(x) - \bar{r}_i(x) \\ \vdots \\ r_{in}(x) - \bar{r}_i(x) \end{bmatrix}.$$

Note that the disagreement blocks sum to zero:

$$(\mathbf{1}^\top \otimes I_d)\, r_i^{\text{dis}}(x) = \sum_{j=1}^{n} \big(r_{ij}(x) - \bar{r}_i(x)\big) = 0.$$

Therefore we have the orthogonal decomposition:

$$r_i(x) = \underbrace{\mathbf{1} \otimes \bar{r}_i(x)}_{\text{consensus}} + \underbrace{\Big(r_i(x) - \mathbf{1} \otimes \bar{r}_i(x)\Big)}_{\text{disagreement}}.$$

Under Assumption 1, the row graph $\mathbb{G}_i$ is connected and bidirectional, hence its Laplacian L^i satisfies

$$\ker(L^i) = \text{span}\{\mathbf{1}\}, \qquad \text{range}(L^i) = \big(\text{span}\{\mathbf{1}\}\big)^\perp.$$

With $\bar{L}^i := L^i \otimes I_d$, we have (1) $\bar{L}^i(\mathbf{1} \otimes v) = 0$ for any $v \in \mathbb{R}^d$, i.e., the Laplacian term cannot change consensus components; and (2) $\text{range}(\bar{L}^i) = \text{range}(L^i) \otimes \mathbb{R}^d$ equals the set of stacked vectors whose blocks sum to zero, i.e., vectors of the same type as $r_i^{\text{dis}}(x)$. Thus, $\bar{L}^i z_i$ can represent (and cancel) disagreement components.
Recall the lifted row objective

$$\min_{z_i} \; \| r_i(x) - \bar{L}^i z_i \|^2.$$

Since $\bar{L}^i z_i$ lies in the disagreement subspace, optimizing over z_i removes as much disagreement as possible from $r_i(x)$. In fact, the minimizer $z_i^*(x)$ makes

$$r_i(x) - \bar{L}^i z_i^*(x) \;=\; \mathbf{P}_{\text{con}}\, r_i(x),$$

where $\mathbf{P}_{\text{con}} := \frac{1}{n}(\mathbf{1}\mathbf{1}^\top \otimes I_d)$ is the projection onto consensus subspace. Note that

$$\mathbf{P}_{\text{con}}\, r_i(x) = \mathbf{1} \otimes \bar{r}_i(x) = \mathbf{1} \otimes \frac{1}{n}(A_i x - b_i),$$

so minimizing over z_i makes row objective exactly proportional to the original row residual $A_i x - b_i$. This is the key motivation of introducing z_i and matches the intuition of lemma 1.

2.4.2 Special Cases

We now analyze two degenerate cases, $n = 1$ (single column) and $m = 1$ (single row). These cases clarify the distinct roles of column communication (consensus + gradient tracking for replicated x-copies) and row communication (the Laplacian-coupled messenger variable z that coordinates a column-partition subproblem inside each row).

Case 1: $n = 1$ (single column)

When $n = 1$, there is only one block variable $x_1 \in \mathbb{R}^d$. Each row $i \in \{1, \ldots, m\}$ has one agent $[\![i1]\!]$ holding (A_{i1}, b_{i1}) and a local copy $x_i := x_{i1}$ of the global variable x. The goal is the standard distributed least-squares problem

$$\min_{x \in \mathbb{R}^d} \sum_{i=1}^{m} f_i(x), \qquad f_i(x) := \|A_{i1}x - b_{i1}\|^2,$$

implemented via local copies $\{x_i\}$ and a consensus mechanism on the column graph $\mathbb{G}$. In this case, the row graphs $\mathbb{G}_i$ have a single vertex, hence $L^i = 0$ and $\bar{L}^i = 0$. Therefore, the messenger variable z_{i1} disappears from the objective and all row-communication steps become vacuous. The proposed method reduces to a GT+Adam-type method over the single column graph $\mathbb{G}$.

A detailed description for $n = 1$ is presented as follows.

Initialization. Each agent $[\![i1]\!]$ chooses an arbitrary $x_i(0) \in \mathbb{R}^d$. It initializes Adam moments $a_{x,i}(0) = 0, b_{x,i}(0) = 0$. It initializes the tracker by one local gradient evaluation: $y_i(0) := \nabla f_i(x_i(0)) = g_i(0)$.

Iteration. At each time $t = 0, 1, 2, \ldots$, every agent $[\![i1]\!]$ performs:

1. **Column Communication (only):**
   (a) **Transmitting:** Agent $[\![i1]\!]$ sends $(x_i(t), y_i(t))$ to its neighbors $\{[\![k1]\!] : k \in \mathcal{N}_i\}$ on $\mathbb{G}$ and receives $\{(x_k(t), y_k(t))\}_{k \in \mathcal{N}_i}$.
   (b) **Updating:** Agent $[\![i1]\!]$ updates Adam moments and the bias-corrected stepsize $a_{x,i}(t+1), b_{x,i}(t+1), \eta(t+1)$ according to (18) and (20), then updates $x_i(t+1)$ according to (22). After computing the new local gradient $g_i(t+1) := 2A_{i1}^\top(A_{i1}x_i(t+1) - b_{i1})$, it finally updates $y_i(t+1)$ according to (23).

Interpretation. In the degenerate case $n = 1$, the z-system plays no role (since $L^i = 0$). The remaining mechanism is exactly the column layer: consensus on $\{x_i\}$ over $\mathbb{G}$ together with gradient tracking so that each agent follows an (asymptotically) unbiased estimate of the global gradient $\nabla \sum_i f_i(x)$, with Adam providing coordinate-wise adaptive scaling.

Case 2: $m = 1$ (single row)

When $m = 1$, there is only one row $i = 1$ and thus no replication across rows. Hence the consensus/mixing component in the x-update over the column graph disappears, but the Adam-style descent step in x remains. In addition, the gradient tracker becomes redundant and telescopes to the exact local gradient. The algorithm reduces to a coupled Adam method on (x, z) for the lifted objective

$$f(x, z) = \|\bar{A}_1 x - \bar{b}_1 - \bar{L}^1 z\|^2 = \sum_{j=1}^{n} \|A_{1j}x_j - b_{1j} - \sum_{k \in \mathcal{N}_{1j}} L^1_{jk} z_k\|^2,$$

where $\bar{L}^1 = L^1 \otimes I_d$ and the coupling across j is realized through the row-neighbor graph $\mathbb{G}_1$.

A detailed description for $m = 1$ is presented as follows.

Initialization. Each agent $[\![1j]\!]$ chooses arbitrary $x_j(0) \in \mathbb{R}^d$ and $z_j(0) \in \mathbb{R}^d$. It initializes Adam moments $a_{x,j}(0) = 0, b_{x,j}(0) = 0, a_{z,j}(0) = 0, b_{z,j}(0) = 0$. It sends $z_j(0)$ to its row neighbors $\{[\![1k]\!] : k \in \mathcal{N}_{1j}\}$ and receives $\{z_k(0)\}_{k\in\mathcal{N}_{1j}}$. It initializes the tracker by one local gradient evaluation: $y_j(0) := \nabla_{x_j} f_{1j}(x(0), z(0)) = g_j^x(0)$.

Iteration. At each time $t = 0, 1, 2, \ldots$, every agent $[\![1j]\!]$ performs:

1. **Row Communication (Only):**
   (a) **Computing:** Agent $[\![1j]\!]$ computes gradients $\left\{\frac{\partial f_{1j}}{\partial z_k(t)}\right\}_{k\in\mathcal{N}_{1j}}$.
   (b) **Transmitting:** Agent $[\![1j]\!]$ sends $\left\{\frac{\partial f_{1j}}{\partial z_k(t)}\right\}_{k\in\mathcal{N}_{1j}}$ to its row neighbors and receives $\left\{\frac{\partial f_{1k}}{\partial z_j(t)}\right\}_{k\in\mathcal{N}_{1j}}$.
   (c) **Updating:** Agent $[\![1j]\!]$ computes the aggregate gradient $h_j(t)$ according to (16), then updates the Adam moments and the bias-corrected stepsize $a_{z,j}(t+1)$, $b_{z,j}(t+1)$, $\eta(t+1)$ according to (19)–(20), respectively, and finally updates $z_j(t+1)$ according to (21).
   (d) **Transmitting:** Agent $[\![1j]\!]$ sends $z_j(t+1)$ to its row neighbors and receives $\{z_k(t+1)\}_{k\in\mathcal{N}_{1j}}$.
2. **Local x-update (Adam descent):**
   (a) **Updating:** Agent $[\![1j]\!]$ first updates $a_{x,j}(t+1), b_{x,j}(t+1), \eta(t+1)$ according to (18) and (20), and then updates

$$x_j(t+1) = x_j(t) - \eta(t+1)\frac{a_{x,j}(t+1)}{\sqrt{b_{x,j}(t+1)} + \epsilon}.$$

   After computing $g_j^x(t+1)$ according to (17) (with $i = 1$), it updates

$$y_j(t+1) = y_j(t) + g_j^x(t+1) - g_j^x(t).$$

   In particular, with $y_j(0) = g_j^x(0)$, it follows by telescoping that $y_j(t) \equiv g_j^x(t)$ for all t.

Interpretation. In the degenerate case $m = 1$, there is no need to reconcile multiple row-copies of the same x_j, so the consensus/mixing part of the x-update disappears. Nevertheless, the Adam-style descent step on x remains the core mechanism for minimizing the lifted objective, while the z-system provides Laplacian-coupled coordination across agents j within the row, removing disagreement components in the stacked residual and enabling a correct row-level least-squares descent.

2.5 Discussion

Our algorithm is a combination of DIGing and Adam. Note that, if we replace the average gradient estimate y_{ij} with the local gradient of x_{ij}, namely $\nabla f_{ij}(x_{ij}(k)$ to update the momentum vector a_x and b_x, then the update rule of x_{ij} becomes 24, and our algorithm degenerates to a DGD style algorithm, but using a local Adam optimizer.

$$a_{x,ij}(t+1) = \beta_1 a_{x,ij}(t) + (1-\beta_1)\nabla f_{ij}(x_{ij}(t)) \tag{24}$$

$$b_{x,ij}(t+1) = \beta_2 b_{x,ij}(t) + (1-\beta_2)(\nabla f_{ij}(x_{ij}(t))^{\odot 2} \tag{25}$$

$$x_{ij}(t+1) = \sum_{k\in\mathcal{N}_i} W_{ik}x_{kj}(t) - \eta(t+1)\frac{a_{x,ij}(t+1)}{\sqrt{b_{x,ij}(t+1)}+\epsilon} \tag{26}$$

If we abandon the Adam optimizer, and use simple gradient descent update rule for both x_{ij} and z_{ij}, then the row partition part of our algorithm degenerates to DIGing, and column partition part becomes simple Gradient Descent:

$$z_{ij}(t+1) = z_{ij}(t) - \eta(t+1)h_{ij}(t+1) \tag{27}$$

$$x_{ij}(t+1) = \sum_{k\in\mathcal{N}_i} W_{ik}x_{kj}(t) - \eta(t+1)y_{ij}(t) \tag{28}$$

$$y_{ij}(t+1) = \sum_{k\in\mathcal{N}_i} W_{ik}y_{kj}(t) + \nabla f_{ij}(x_{ij}(t+1)) - \nabla f_{ij}(x_{ij}(t)) \tag{29}$$

3 Quantum algorithm

3.1 Distributed Quantum Linear System Problem (DQLSP)

We now introduce a distributed quantum algorithm for solving QLSP. First, the bridge bewteen quantum part and classical part is the amplitude embedding of a classical unit vector into a quantum state, or conversely a vector representation of a quantum state. Consider a n-qubits quantum state $|x\rangle$. A classical 2^n dimension unit vector $x = (x_0, x_1, \cdots, x_{2^n-1})^\top$ is encoded into $|x\rangle$ through amplitude embedding:

$$|x\rangle = \sum_{i=0}^{2^n-1} x_i |i\rangle = x_0|0\rangle + x_1|1\rangle + ... + x_{2^n-1}|2^n-1\rangle . \tag{30}$$

where $|i\rangle, i \in [0, 2^n-1]$ are computational basis of the n qubits system. From now on, the vector we consider should have power of 2 dimension. Also for simplicity, we limit our linear system to be real, thus our quantum states live in real Hilbert space instead of the most general complex Hilbert space.

Consider solving a least square solution for the linear system $Ax = b$. In quantum formulation, the vector b should be some known quantum state but unnormalized. Matrix A should be considered

as the Linear Combination of Unitaries (LCU)

$$A = \sum_{l=1}^{L} c_l A_l \tag{31}$$

where A_l are unitaries that can be expressed by certain quantum operators. Solving the systems represents preparing a unnormalized quantum state $|x^*\rangle$ which satisfies:

$$|x^*\rangle \in \arg\min_{|x\rangle \in \mathbb{R}^{2^n}} \| A\,|x\rangle - |b\rangle \|^2 \tag{32}$$

We now use the calligraphic letters to denote normalized quantum state such as $|\mathcal{X}\rangle$ and $|\mathcal{B}\rangle$, and using squared inner products to denote the magnitude of the unnormalized states, such as $\sqrt{\langle x|x\rangle}$, and $|x\rangle = \sqrt{\langle x|x\rangle}\,|\mathcal{X}\rangle$. State $|x\rangle$ is the same as classical vector x, and the ket notation is just to remind us we are in the quantum algorithm. In circuit model of quantum computing, the normalized states can be represented as the output state after a sequence of gates acting on some qubits. Together with some positive parameter that parameterize the magnitude $\sqrt{\langle x|x\rangle}$, we are able to fully represent a classical vector in a quantum way.

For DQLSP, we still suppose a agent network of mn agents and partition the linear system of equations into:

$$\begin{bmatrix} A_{11} & \cdots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \cdots & A_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix},$$

where m and n are two positive integers. Each agent is assigned d qubits such that each A_{ij} is a sparse square submatrix in $\mathbb{R}^{2^d \times 2^d}$ and can be decomposed as $A_{ij} = \sum_l c_l A_{ij}^l$. Vector $|b_{ij}\rangle \in \mathbb{R}^{2^d}$ and $\sum_{j=1}^{n} |b_{ij}\rangle = |b_i\rangle$. The goal of each agent is to compute an $|x_j\rangle^* \in \mathbb{R}^{2^d}$ such that the stacked vector

$$|x\rangle^* = \begin{bmatrix} |x_1^*\rangle \\ |x_2^*\rangle \\ \cdots \\ |x_n^*\rangle \end{bmatrix} \in \mathbb{R}^{n2^d} \tag{33}$$

satisfy the least square solution.

3.2 Variational Ansatz

We now discuss how we prepare our quantum state $|\mathcal{X}\rangle$. A variational algorithm employs a gate sequence $V(\alpha)$ with trainable parameters α on the state $|0\rangle$ to prepare the state $|\mathcal{X}\rangle = V(\alpha)\,|0\rangle$. In general $V(\alpha)$ can be written as

$$V(\alpha) = G_{k_L}(\alpha_L) \ldots G_{k_i}(\alpha_i) \ldots G_{k_1}(\alpha_1). \tag{19}$$

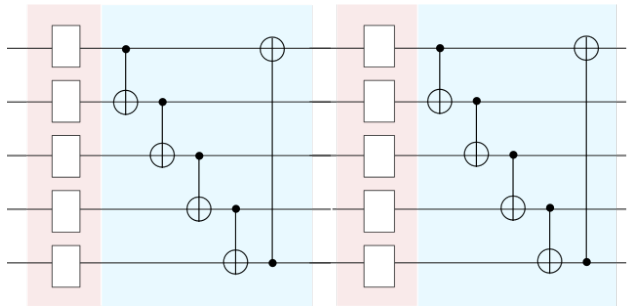

Figure 1: A example of Hardware-Efiicient Ansatz with 2 layers and 4 qubits

Here $k = (k_L, \ldots, k_1)$ identifies the types of gates and their placement in the circuit (i.e., on which qubit they act), while α are continuous parameters. In principle the sequence k is changeable during optimization, but for simplicity our method fix k and optimize over parameter α only, which corresponds to a "fixed-structure" ansatz.

When working on a specific quantum hardware, a popular choice of ansatz will be the so-called Hardware-Efficient Ansatz (HEA). Gate set of this kind of ansatz will only include gates that is naive to a specific hardware, so the number of gates and parameters can be reduced. During simulation, we use an example of HEA: the *BasicentanglementLayer* template in PennyLane, which is composed of single-qubit y-axis rotation gate with trainable angle parameter, and CNOT gate between neighbor wires to introduce entanglement thus enhance expressivity. This also ensures the output states live in the real Hilbert space since y-axis rotation is a real-parameterized operation. A graphical description of a two-layer, four qubits HEA is shown in Fig.1. On real world quantum hardware, the structure of HEA depends on the coupling map and qubit topology.

The choice of ansatz has significant influence on the variational algorithm. A further discussion on the ansatz is Section 3.4.

3.3 Objective funtion

Since we already have a classical optimization algorithm for solving the distributed least square problem, we just need to describe how our local quantum agent deal with the input parameter and how they calculate local objective function and gradients.

The local objective function follows the design of classical local objective function in eq. 13, with all classical variables being represented as a quantum state and its magnitude parameter. Let us denote the ansatz of x and z variable:

$$|\mathcal{X}_{ij}\rangle = V(\alpha_{ij})\,|0\rangle\,, \quad |x_{ij}\rangle = \sigma_{ij}\,|\mathcal{X}_{ij}\rangle \tag{34}$$

$$|\mathcal{Z}_{ij}\rangle = W(\beta_{ij})\,|0\rangle\,, \quad |z_{ij}\rangle = \lambda_{ij}\,|\mathcal{Z}_{ij}\rangle \tag{35}$$

where $\alpha_{ij} \in [-\pi, \pi]^l, \beta_{ij} \in [-\pi, \pi]^l, \sigma_{ij} \in \mathbb{R}^+, \lambda_{ij} \in \mathbb{R}^+$ are all local trainable parameters. For simplicity, we assume all local b_{ij} is a unit vector, and can be represented by a unitary acting on

ground input state:$|b_{ij}\rangle = U_{ij}|0\rangle$. We additionally introduce a state $|s_{ij}\rangle$ which includes all the trainable variable, differ from static $|b_{ij}\rangle$

$$|s_{ij}\rangle = \sigma_{ij} A_{ij} |\mathcal{X}_{ij}\rangle - \sum_{k\in\mathcal{N}_{ij}} L^i_{jk}\lambda_{ik} |\mathcal{Z}_{ik}\rangle \tag{36}$$

Then, our local objective function can be expressed as:

$$\begin{aligned} f_{ij} &= \big\| \, |s_{ij}\rangle - |b_{ij}\rangle \, \big\|^2 \\ &= \langle s_{ij}|s_{ij}\rangle + \langle b_{ij}|b_{ij}\rangle - 2\mathrm{Re}(\langle b_{ij}|s_{ij}\rangle) \end{aligned} \tag{37}$$

The involving terms can be expanded as follows:

$$\begin{aligned} \langle s_{ij}|s_{ij}\rangle = \sigma_{ij}^2 &\langle \mathcal{X}_{ij}| A^\dagger_{ij} A_{ij} |\mathcal{X}_{ij}\rangle \\ &+ \sum_{k\in\mathcal{N}_{ij}} \sum_{p\in\mathcal{N}_{ij}} L^i_{jk} L^i_{jp} \lambda_{ik}\lambda_{ip} \langle \mathcal{Z}_{ik}|\mathcal{Z}_{ip}\rangle \\ &- 2\sigma_{ij} \sum_{k\in\mathcal{N}_{ij}} L^i_{jk}\lambda_{ik} \mathrm{Re}\big(\langle \mathcal{X}_{ij}| A^\dagger_{ij} |\mathcal{Z}_{ik}\rangle \big) \end{aligned} \tag{38}$$

$$\langle b_{ij}|s_{ij}\rangle = \sigma_{ij} \langle b_{ij}| A_{ij} |\mathcal{X}_{ij}\rangle - \sum_{k\in\mathcal{N}_{ij}} L^i_{jk}\lambda_{ik} \langle b_{ij}|\mathcal{Z}_{ik}\rangle \tag{39}$$

Using the variational ansatz $V(\alpha)$ and $W(\beta)$, these correspond to:

$$\langle \mathcal{X}_{ij}| A^\dagger_{ij} A_{ij} |\mathcal{X}_{ij}\rangle = \langle 0| V^\dagger(\alpha_{ij}) A^\dagger_{ij} A_{ij} V(\alpha_{ij}) |0\rangle \tag{40}$$

$$\langle \mathcal{X}_{ij}| A^\dagger_{ij} |\mathcal{Z}_{ik}\rangle = \langle 0| V^\dagger(\alpha_{ij}) A^\dagger_{ij} W(\beta_{ik}) |0\rangle \tag{41}$$

$$\langle \mathcal{Z}_{ik}|\mathcal{Z}_{ip}\rangle = \langle 0| W^\dagger(\beta_{ik}) W(\beta_{ip}) |0\rangle \tag{42}$$

$$\langle b_{ij}| A_{ij} |\mathcal{X}_{ij}\rangle = \langle 0| U^\dagger_{ij} A_{ij} V(\alpha_{ij}) |0\rangle \tag{43}$$

$$\langle b_{ij}|\mathcal{Z}_{ik}\rangle = \langle 0| U^\dagger_{ij} W(\beta_{ik}) |0\rangle \tag{44}$$

The widely-used Hadamard Test can be used to measure all the term above. However, the naive use of Hadamard Test will introduce controlled-gates with variational parameters, which may result in heavy overhead on real quantum computers. Instead, we build a circuit with post-selection method to evaluate these terms, except for eq.40 which can be evaluated through Hadamard-Overlap Test more efficiently. One example of circuit for evaluating term in eq.43 is in Fig.2. What we measure is the probability of finding the system in the ground state $|0\rangle$ (given the ancillary qubit measured in $|0\rangle$).

Now, we are able to evaluate the global objective function in eq.14. Also, the gradient of every variational parameter can be calculated using the same circuit according to [3]. This is also known as the parameter-shift rule. The gradient of parameter σ_{ij} and λ_{ij} are easy to calculate since they are linear or quadratic in the global objective function.

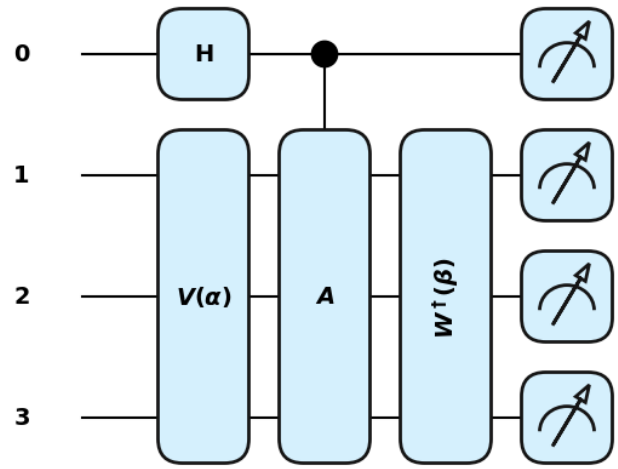

Figure 2: The circuit for evaluating eq.43

To be more closely related to our classical algorithm, we absorb parameter σ_{ij} into α_{ij} to form a new $\boldsymbol{\alpha_{ij}} = (\sigma_{ij}, \alpha_{ij})$ since they together maps to variable x_{ij} in classical algorithm. Samely, λ_{ij} is absorbed into β_{ij} to form $\boldsymbol{\beta_{ij}} = (\lambda_{ij}, \beta_{ij})$. Now we have our variable x_{ij} and z_{ij} fully represented by parameter $\boldsymbol{\alpha_{ij}}$ and $\boldsymbol{\beta_{ij}}$, meanwhile the local objective function changes its input: $f_{ij} = f_{ij}(\boldsymbol{\alpha_i}, \boldsymbol{\beta_i})$ compared with eq.14. The gradient variable is denoted as:

$$h_{ij}(t) := \nabla_{\boldsymbol{\beta_{ij}}} f = \sum_{k \in \mathcal{N}_{ij}} \nabla_{\boldsymbol{\beta_{ij}}} f_{ik}(\boldsymbol{\alpha_i}(t), \boldsymbol{\beta_i}(t)) \tag{45}$$

$$g_{ij}^{\boldsymbol{\alpha}}(t) := \nabla_{\boldsymbol{\alpha_{ij}}} f_{ij}(\boldsymbol{\alpha_i}(t), \boldsymbol{\beta_i}(t)) \tag{46}$$

The update rule of quantum algorithm is exactly same as the classical algorithm in Section 2.3, except that all classical variable are translated into their corresponding variational parameters. The gradient tracking technique and Adam optimizer now act on these variational parameters. All parameters are randomly initialized.

3.4 Discussion

In this part we may be able to heuristically explain the advantage of our distributed design over traditional VQLS. I borrow two concepts from Sim.et.al.

3.4.1 Expressibility

Expressibility is defined as a circuit's ability to generate (pure) states that are well representative of the Hilbert space. One way to compute this notion is to compare distribution of states obtained from sampling over parameter space to a random Haar state. The final measure is the KL divergence between the estimated fidelity distribution and that of the Haar-distributed ensemble. We can do a simple simulation to sample this and compare.

3.4.2 Entanglement capability

This part uses Meyer-Wallach (MW) entanglement measure to determine the ability of certain circuit to generate entanglement. Our heuristic proof can follow this story.

Assumption 1: A solution state $|x\rangle$ with high entanglement can be expressed by a circuit *only if* the circuit has high entanglement capacity.

Assumption 2: For most variational ansatz, more depth is need for achieving higher entanglement capacity.

Lemma 3. *Consider using a 2×2 agent network to solve a linear system whose solution is proportional to the vector representation of n-qubits quantum state $|x\rangle$, then the partition of the solution is equvialent to the following decomposition:*

$$|x\rangle = \alpha |0\rangle \otimes |u\rangle + \beta |1\rangle \otimes |v\rangle \tag{47}$$

where $|u\rangle$ and $|v\rangle$ are both (n-1)-qubits state, and α and β are normalization coefficients.

Lemma 4. *The Meyer-Wallach entanglement measure of state $|u\rangle$ and $|v\rangle$ is no more than state $|x\rangle$.*

4 Simulation results

Here we show the result of the simulation that check the necessity of the three steps we used in the algorithm. The combination of techniques is summarized in Table 1, and all combinations are run under same conditions to compare. Specifically, for combination C in Table 1, the update rule for messenger z is eq.27, and update rule for variable x remains to be eq.**??**.

For the simulation setup, we use 4×4 agent network, with each block being a 4×4 submatrix. We pick three linear system for solving, with the condition number of A is $196, 100, 12$. We also do a parameter scan on three different initial stepsize, and pick the best result for later discussion.

Each experiment is run for 10^4 epoches. Although reaching precision 10^{-2} is considered a true criteria, it take really long time for condition number $\kappa = 196$ example. In order that the final difference does not reach 10^{-2}, here is the substitute criteria of necessity:

Combination	Technique for row partition (communicate with column neighbor)	Technique for column partition (communicate with row neighbor)
A	tracking + Adam	Adam
B	tracking + Gradient Descent	Adam
C	tracking + Adam	Gradient Descent
D	tracking + Gradient Descent	Gradient Descent

Table 1: Combination of techniques

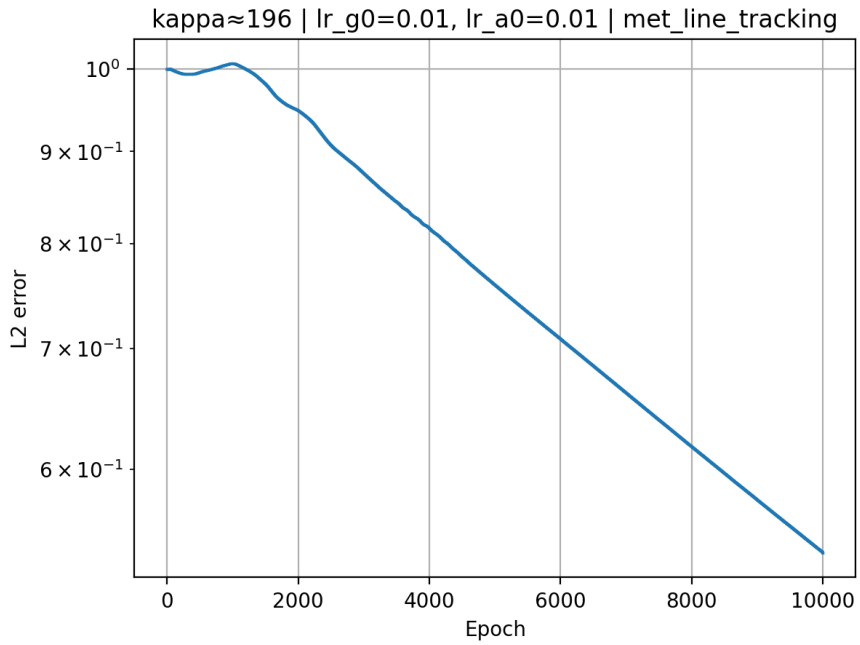

Figure 3: History of the difference between estimated solution and true solution after 10^4 iterations. Using combination A in Table1.

1. The difference between estimated solution and true solution can significantly decrease after 10^4 iterations.

2. The difference exhibits a approximately steady linear decrease on logarithmic scale.

For $\kappa = 196$ example, Combination A surpass others in this example as the only one that pass the above substitute criteria. The detailed comparison of the history of A and C is shown below in Fig.3 and Fig.4. However, for the $\kappa = 12$ case, all the combinations can passes the substitute criteria. This suggests that all the combinations make sense in theory, but taking consideration of real simulation and efficiency, combination A is the most robust.

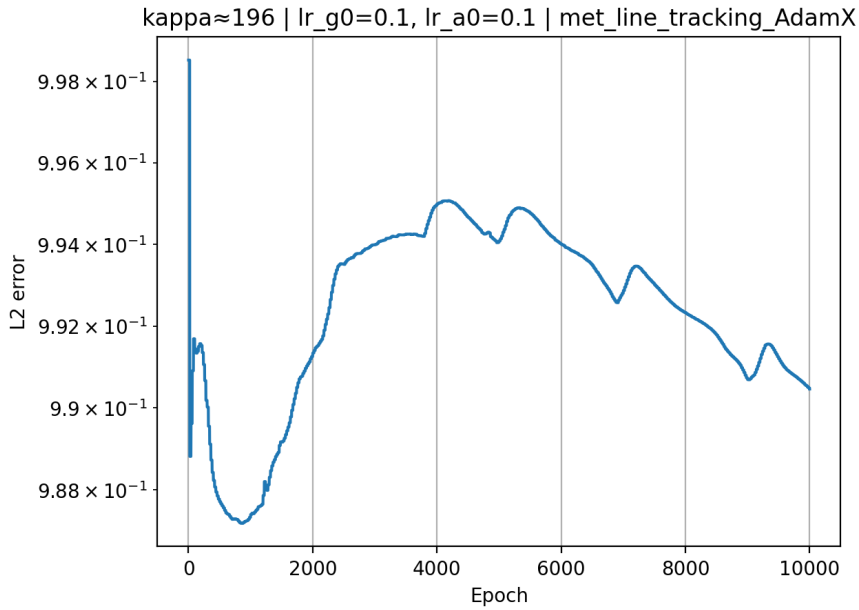

Figure 4: History of the difference between estimated solution and true solution after 10^4 iterations. Using combination C in Table1.

References

[1] Angelia Nedić, Alex Olshevsky, and Wei Shi. Achieving geometric convergence for distributed optimization over time-varying graphs. *SIAM Journal on Optimization*, 27(4):2597–2633, 2017.

[2] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 12 2014.

[3] Carlos Bravo-Prieto, Ryan LaRose, M. Cerezo, Yigit Subasi, Lukasz Cincio, and Patrick J. Coles. Variational Quantum Linear Solver. *Quantum*, 7:1188, November 2023.